National University of Computer and Emerging Sciences

Karachi Campus

Database Systems (CS2005)

Final Exam

Date: Monday, 15th Dec 2025

Time: 09:00 am to 12:00 pm

Course Instructor(s): Dr. Zulfiqar Ali Memon, Dr. Farrukh Salim, Ms. Atiya Jokhio, Ms. Javeria Farooq, Mr. Basit Ali, Ms. Hajira Ahmed, Mr. Ali Naseer, Mr. Huzaifa, Mr. Umair

Total Time (Hrs):	**3**
Total Marks:	**50**
Total Questions:	**8**

Do not write below this line

Attempt all the questions.

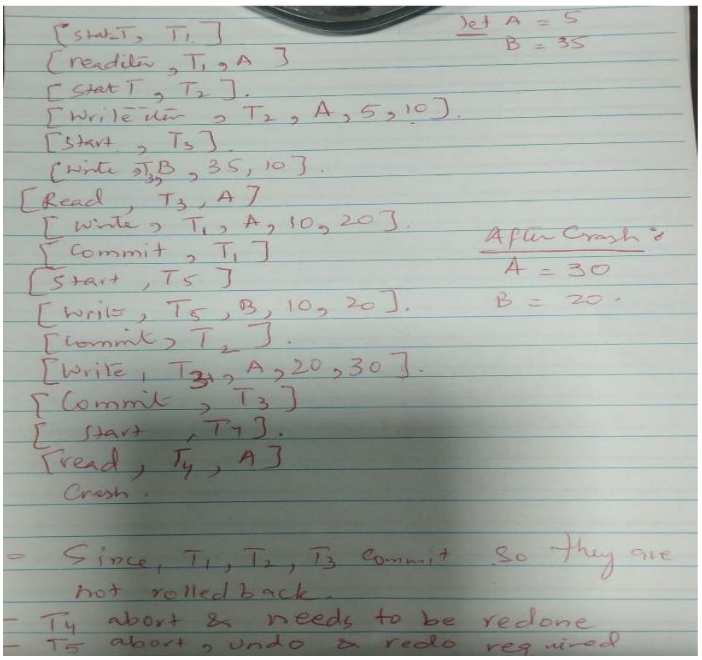
CLO # 2: Analyze an information storage problem and derive an information model expressed in the form of an entity relation diagram and other optional analysis forms, such as a data dictionary.

Q1: [marks: 5] [Estimated Time: 20 minutes]

Assume that immediate update recovery is being used in a DBMS. A schedule at system crash is shown that includes five transactions. **Initial Values: A=5, B=35.**

T1	T2	T3	T4	T5
R(A)				
	A = 10			
	W(A)			
		B = 10		
		W (B)		
		R(A)		
A = A + 10				
W(A)				
commit				
				B = 20
				W(B)
	commit			
		A = A + 10		
		W(A)		
		commit		
			R(A)	
			A = A + 15	
**********System Crash**********				

1. Design the log for the schedule in given Figure **[2]**
2. What are values after the crash for A and B. **[2]**
3. Specify which transactions need to undo? Provide justification for all the transactions, why do they need to undo or not. **[1]**



CLO # 2: Analyze an information storage problem and derive an information model expressed in the form of an entity relation diagram and other optional analysis forms, such as a data dictionary.

Q2: [marks: 4] [Estimated Time: 20 minutes]

Consider the following scenario below and answer the following parts:
An online food delivery platform handles thousands of simultaneous orders. Each order triggers parallel updates to inventory, rider assignment tables, and customer wallets. The system uses locking for consistency, but due to the heavy concurrency, the platform faces deadlocks several times per hour. Although the transactions are short, the deadlocks cause minor delays.

a) Briefly explain one common Deadlock Prevention technique you would use. **[2]**
b) Based on the scenario (high concurrency, small transactions, frequent deadlocks), should the database use Deadlock Prevention or Deadlock Detection & Recovery? Justify your answer by comparing the overhead and efficiency of both approaches in this environment**. [2]**

Solution:

A) One Deadlock Prevention Technique – *Wound-Wait Scheme*

- In the Wound-Wait scheme, each transaction is assigned a timestamp.
- If an older transaction requests a lock held by a younger one, it *wounds* (forces rollback of) the younger transaction.
- If a younger transaction requests a lock held by an older one, it must wait. This avoids circular wait and prevents deadlocks.

B) Recommended Approach

- Deadlock Detection & Recovery is recommended.
- Prevention techniques (like Wound-Wait) constantly compare timestamps and may abort transactions unnecessarily, creating extra overhead.
- In a high-concurrency system with short transactions, forced rollbacks from prevention cause more interruption than necessary.
- Detection & Recovery allows maximum concurrency: transactions proceed normally, and only actual deadlocks are handled by aborting one short transaction.
- Since restarts are cheap, detection gives higher throughput and avoids the restrictive behavior of prevention.
- Detection & Recovery provides better performance and scalability for this fast, high-volume environment.

CLO # 2: Analyze an information storage problem and derive an information model expressed in the form of an entity relation diagram and other optional analysis forms, such as a data dictionary.

Q3: [marks: 5] [Estimated Time: 20 minutes]

Part 1: Consider two transactions given below and explain why this blocking happens with binary locks and how S/X locks would improve the situation. **[2.5]**

- T1 reads item D and holds a binary lock.
- T2 wants to write D but keeps waiting because T1 is still reading.

Solution:

Binary locks treat all operations the same (read = write), so writer T2 must wait even if T1 is only reading. With S/X locks:

- **T1 would hold S-lock on D (shared)**
- **T2 would request X-lock, and it must wait, but**
- **If multiple readers existed, they could share S-locks, increasing concurrency. Thus, S/X locks give better concurrency by allowing multiple readers.**

Part 2: In below series of transactions, does T1 follow Strict 2PL, Basic 2PL, or none? Explain briefly. **[2.5]**

- T1: S-lock(A) → reads A → X-lock(B) → writes B
- T2: S-lock(A) → waits
- T1: unlock(A) → unlock(B)
- T2: S-lock(A) granted → reads A

Solution:

T1 acquires all locks before releasing any → satisfies Basic 2PL.
However, Strict 2PL requires X-locks to be held until commit. T1 releases X(B) before commit point, so Strict 2PL is NOT followed.

CLO # 1: Explain fundamental database concepts.
Q4: [marks: 6] [Estimated Time: 25 minutes]

Imagine a cafe management system where customers place food orders, update their table reservations, and process bills concurrently. During a busy lunch hour, multiple transactions occur at the same time:

a) Customer A places an order for a sandwich and juice, but the system crashes before the order is fully saved. **[2]**

- **Atomicity:**
- **Atomicity ensures that a transaction must either complete entirely or not execute at all.**
- **Since the system crashed before the order was fully saved, the whole "place order" transaction is rolled back.**
- **Customer A's order will *not* appear partially (e.g., showing only a sandwich but not the juice).**

b) Customer B orders the last available slice of cheesecake. The system deducts the cheesecake from inventory, but before this deduction is committed, Customer C views the dessert menu on the tablet. [2]

Isolation ensures that concurrent transactions do not affect each other until they are committed. Customer C must **not** see the uncommitted deduction made by Customer B's order. Therefore, Customer C's menu view will still show the cheesecake available until Customer B's transaction is successfully committed.

c) Customer D pays the bill via credit card. The bank successfully processes the payment, but immediately afterward the cafe's billing system crashes, and the payment status is not updated in the café records. **[2]**

Durability and Consistency

- Once a transaction (payment) is committed, it must **survive system failures**.
- The café system must use logs or backups so that after recovery, it restores the *committed payment* status.
- The system must ensure valid and correct data states before and after the transaction.

Explain how the **ACID properties** would handle the above scenarios and ensure reliable and correct cafe-management transaction processing.

CLO # 2: Analyze an information storage problem and derive an information model expressed in the form of an entity relation diagram and other optional analysis forms, such as a data dictionary.

Q5: [marks: 10] [Estimated Time: 30 minutes]

Consider the following 10 schedules of two transactions each. For each schedule, show the proper working and classify if it is: **recoverable, non recoverable, cascading rollback, cascadeless, strict, or conflict serializable.** A schedule may have more than one property. R1 defines read of transaction 1, W1 defines write of transaction 1, A1 defines abort of transaction 1, and C1 defines commit of transaction 1. Same goes for transaction 2.

1. R1(X); R2(X); W1(X); C1; W2(X); C2

   T2 reads X **before T1 writes X** → no read-from dependency.
   T2 writes X **after T1 commits** → safe.

Recoverable: Yes (no T2 commit depends on T1)
Cascadeless: Yes(no reads of uncommitted writes)
Strict: yes (T2 writes X only after T1 commits)
Conflict Serializable: Yes→ order: **T1 → T2**

2. R2(Y); W2(Y); R1(Y); W1(Y); C1; C2

- T1 reads Y after T2 writes Y → **T1 depends on T2**.
- T1 commits before T2 → **BAD**.
- **Non-recoverable:** No (T1 commits before T2)
- **Cascading Rollback:** Yes (T1 reads uncommitted T2 data)
- **Strict:** Yes (R1(Y) while W2(Y) uncommitted)
- **Conflict Serializable:** Yes→ order: **T2 → T1**

3. R1(X); W1(X); R2(Y); W2(Y); C2; C1
   No reading of each other's writes.

- **Recoverable:** Yes
- **Cascadeless:** Yes
- **Strict:** Yes
- **Conflict Serializable:** Yes (no conflicts)

4. R2(X); R1(X); W2(X); A2; W1(X); C1

- **Recoverable:** Yes (T1 not reading T2 writes)
- **Cascadeless:** Yes (no reads of uncommitted writes)
- **Strict:** yes (T1 waits until T2 abort before writing X)
- **Conflict Serializable:** Yes → order: **T2 → T1**

5. W1(X); R2(X); W2(X); C2; C1

- **Non-recoverable:** No (T2 commits before T1)
- **Cascading Rollback:** Yes (T2 read uncommitted write)
- **Strict:** No (R2(X) allowed while T1 uncommitted)
- **Conflict Serializable:** Yes → order **T1 → T2**

6. R2(X); W2(X); R1(X); A2; A1

- **Recoverable:** Yes (T1 does not commit before T2)
- **Cascading Rollback:** Yes (T1 reads uncommitted write of T2)
- **Strict:** No (R1(X) while W2(X) uncommitted)
- **Conflict Serializable:** Yes → order: **T2 → T1**

7. W2(Y); R1(Y); W1(Y); C1; A2

- **Non-recoverable:** No (T1 commits before T2)
- **Cascading Rollback:** Yes (T1 reads uncommitted write)
- **Strict:** No
- **Conflict Serializable:** Yes → order: **T2 → T1**

8. R1(X); R2(X); W2(X); C2; W1(X); C1

- **Recoverable:** Yes
- **Cascadeless:** Yes
- **Strict:** No
- **Conflict Serializable:** No

9. W1(X); W2(X); R1(X); C1; C2

- **Non-recoverable:** No
- **Cascading Rollback:** Yes (R1(X) reads uncommitted W2)
- **Strict:** No
- **Conflict Serializable:** yes → order: **T2 → T1**

10. R2(X); W2(X); R1(Y); W1(Y); C2; C1

Non-recoverable: No
Recoverable: Yes
Cascading Rollback: No
Cascadeless: Yes
Strict: Yes
Conflict Serializable: Yes

CLO # 3: Demonstrate an understanding of normalization theory to normalize the database and formulate, using SQL & relational algebra, solutions to a broad range of query & data problems in a team work..

Q6: [marks: 10] [Estimated Time: 30 minutes]

Athletes, who are members of teams, compete in running events, which are held at fixtures throughout the year. primary keys are underlined and the foreign keys you have to identify yourself.

Athlete (AthleteID, Surname, Forename, DateOfBirth, Gender, TeamName)
EventType (EventTypeID, Gender, Distance, AgeGroup)
Fixture (FixtureID, FixtureDate, LocationName)
EventAtFixture (FixtureID, EventTypeID)
EventEntry (FixtureID, EventTypeID, AthleteID)

Using the above schema, Write down the SQL queries and relational algebra for the following questions

1) List all athletes with the events they entered, including fixture location
2) Count how many events each athlete entered
3) Count number of athletes in each event at each fixture
4) Show all teams and the number of athletes per team who entered events in 2018
5) Find events (distance + age group) that appear in more than 3 fixtures

Solution:

```
Q1. SELECT
    A.AthleteID,
    A.Forename,
    A.Surname,
    ET.Distance,
    ET.AgeGroup,
    F.LocationName,
    F.FixtureDate
FROM EventEntry EE
JOIN Athlete A ON EE.AthleteID = A.AthleteID
JOIN EventType ET ON EE.EventTypeID = ET.EventTypeID
JOIN Fixture F ON EE.FixtureID = F.FixtureID;
π AthleteID, Forename, Surname, Distance, AgeGroup, LocationName, FixtureDate
(
  ( EventEntry ⋈_{EventEntry.AthleteID = Athlete.AthleteID} Athlete )
  ⋈_{EventEntry.EventTypeID = EventType.EventTypeID} EventType
  ⋈_{EventEntry.FixtureID = Fixture.FixtureID} Fixture
)

Q2
SELECT
    A.AthleteID,
    A.Forename,
    A.Surname,
    COUNT(*) AS TotalEventsEntered
FROM EventEntry EE
JOIN Athlete A ON EE.AthleteID = A.AthleteID
```

```
GROUP BY A.AthleteID, A.Forename, A.Surname
HAVING COUNT(*) > 1;        -- Only show athletes who entered more than 1 event

σ TotalEventsEntered > 1
(  γ_{Athlete.AthleteID, Forename, Surname; count(*) -> TotalEventsEntered}  (
   EventEntry ⋈_{EventEntry.AthleteID = Athlete.AthleteID} Athlete  ))

Q3
SELECT
   F.LocationName,
   F.FixtureDate,
   ET.Distance,
   ET.AgeGroup,
   COUNT(EE.AthleteID) AS TotalAthletes
FROM EventEntry EE
JOIN Fixture F ON EE.FixtureID = F.FixtureID
JOIN EventType ET ON EE.EventTypeID = ET.EventTypeID
GROUP BY
   F.LocationName,
   F.FixtureDate,
   ET.Distance,
   ET.AgeGroup
HAVING COUNT(EE.AthleteID) >= 5;
σ TotalAthletes >= 5
(
 γ_{LocationName, FixtureDate, Distance, AgeGroup; count(AthleteID) -> TotalAthletes}
 (
  ( EventEntry ⋈_{EventEntry.FixtureID = Fixture.FixtureID} Fixture )
  ⋈_{EventEntry.EventTypeID = EventType.EventTypeID} EventType
 )
)

Q4
SELECT
   A.TeamName,
   COUNT(DISTINCT A.AthleteID) AS TotalAthletesIn2018
FROM EventEntry EE
JOIN Athlete A ON EE.AthleteID = A.AthleteID
JOIN Fixture F ON EE.FixtureID = F.FixtureID
WHERE YEAR(F.FixtureDate) = 2018
GROUP BY A.TeamName
HAVING COUNT(DISTINCT A.AthleteID) > 0;
σ FixturesIn2018
(
 — first join EventEntry, Athlete and Fixture
 ( EventEntry ⋈_{EventEntry.AthleteID = Athlete.AthleteID} Athlete )
 ⋈_{EventEntry.FixtureID = Fixture.FixtureID} Fixture
)

Q5
SELECT
   ET.Distance,
   ET.AgeGroup,
   COUNT(DISTINCT F.FixtureID) AS FixturesHeld
FROM EventAtFixture EF
```

```
JOIN EventType ET ON EF.EventTypeID = ET.EventTypeID
JOIN Fixture F ON EF.FixtureID = F.FixtureID
GROUP BY ET.Distance, ET.AgeGroup
HAVING COUNT(DISTINCT F.FixtureID) > 3;
σ TotalAthletesIn2018 > 0
(
 γ_{TeamName; count_distinct(AthleteID) -> TotalAthletesIn2018}
 (
  σ (FixtureDate >= '2018-01-01' ∧ FixtureDate <= '2018-12-31')
  (
   ( EventEntry ⋈_{EventEntry.AthleteID = Athlete.AthleteID} Athlete )
   ⋈_{EventEntry.FixtureID = Fixture.FixtureID} Fixture
  )
 )
)
```

CLO # 3: Demonstrate an understanding of normalization theory to normalize the database and formulate, using SQL & relational algebra, solutions to a broad range of query & data problems in a team work..

Q7: [marks: 5] [Estimated Time: 20 minutes]

Consider the following relation
Project_Grade (**Project_id, Student_id,** student_name, project_title, grade, submission_date, bonus)
The given FD's of above relation are:

FD1: student_id--> student_name
FD2: project_id-->project_title
FD3: submission_date--> bonus marks

Note: If any attribute is not mentioned in any FD, then by default they will be dependent on the whole primary key.

Identify partial and transitive dependencies (if any) and convert the table to 3NF and

i) show all relation schema in 2NF. **[2.5]**
ii) show all relation schema in 3NF. **[2.5]**

Solution:

i) FD1 and FD2 cause partial dependencies so not in 2NF.

Normalizing to 2NF:
Project_Grade(Project_id, Student_id, grade, submission_date, bonus)
Student(Student_id ,student_name)
Project(Project_id, project_title)

ii) FD3 causes transitive dependency.

Normalizing to 3NF.
Student(Student_id ,student_name)
Project(Project_id, project_title)
Project_Grade(Project_id, Student_id, grade, submission_date)
Bonus_Info(submission_date, bonus)

CLO # 2: Analyze an information storage problem and derive an information model expressed in the form of an entity relation diagram and other optional analysis forms, such as a data dictionary.

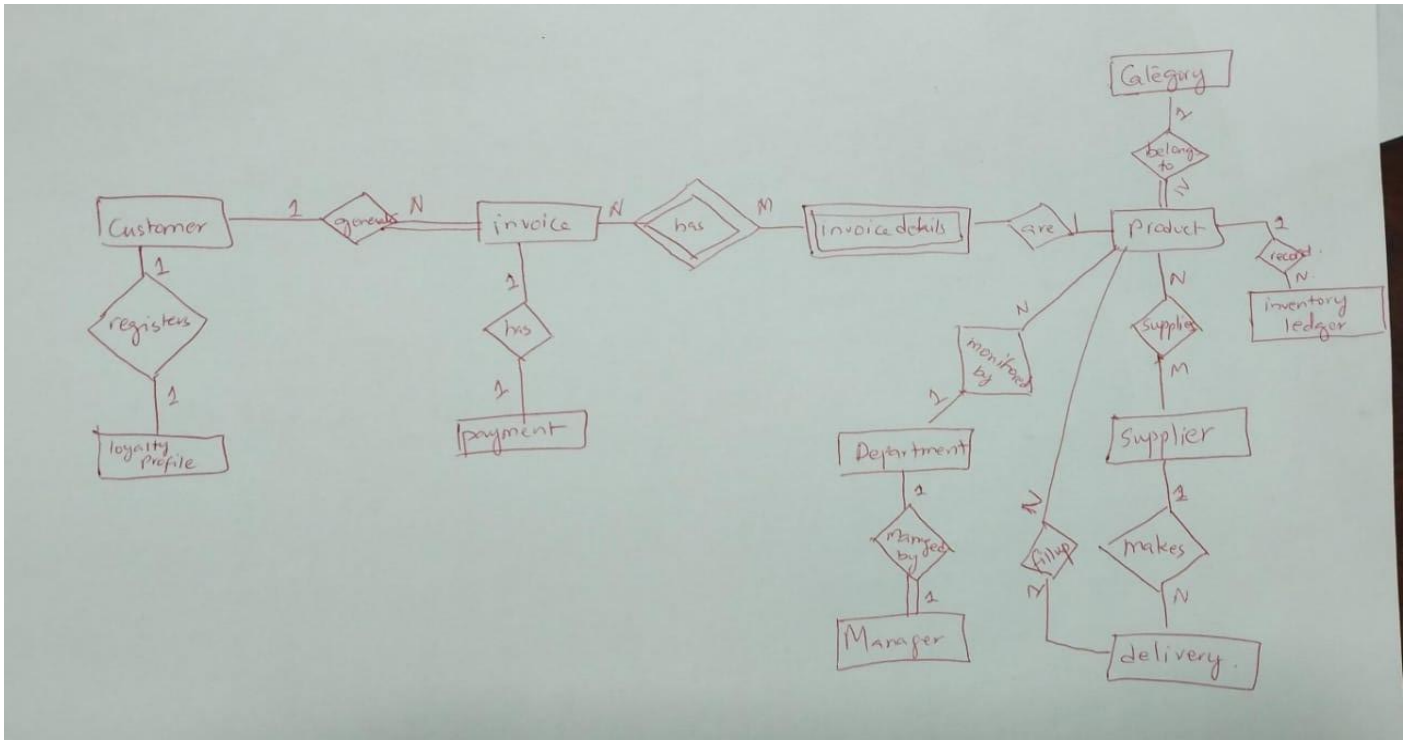
Q8: [marks: 5] [Estimated Time: 25 minutes]

A super mart manages thousands of products, multiple departments, daily customer purchases, inventory tracking, suppliers, and billing operations. Customers can register with unique account IDs and maintain optional loyalty profiles that store reward points and purchase history. Products are organized into categories such as groceries, household items, fresh produce, dairy, and electronics. Each product has attributes like product ID, name, brand, price, category, and stock level. Suppliers, identified by unique supplier IDs, provide products to the super mart. Each supplier delivers multiple products, and each product may be supplied by more than one supplier. Supplier deliveries include delivery ID, date, quantity, and cost, contributing directly to inventory replenishment.

Customers purchase products through invoices/bills that contain invoice number, date, time, total amount, and payment method. Each invoice includes multiple products with details such as quantity, price per unit, and subtotal. Payments can be processed through cash, credit/debit cards, or mobile wallets, and each payment record contains payment ID, method, amount, timestamp, and status.

Inventory in each department (e.g., dairy, fruit & vegetable section, bakery, electronics aisle) is monitored in real time. Each department has a manager responsible for product arrangement and stock supervision. The system maintains stock ledger entries showing stock-in and stock-out records with timestamps.

Promotional campaigns and discount offers are applied to selected products, with details like offer ID, discount percentage, validity period, and applicable categories.The super mart ensures smooth functioning across procurement, inventory management, billing, payment processing, and customer service. **Your task is to create an Entity Relationship Diagram to show the complete flow of the platform. Also mention the structural constraints if required.**



Good Luck!